IFSP · LABORATÓRIO DE PROGRAMAÇÃO 1

Mercado Viva.

Do terminal à URL de avaliação

Guia de instalação, banco de dados, GitHub e Vercel.

Python · Flask · MVC · MPA · SQLAlchemy
Dois temas visuais · PostgreSQL persistente
Caixa, estoque, compras e financeiro simulados

Professor Celso Gonsalez Saes
Projeto final · Roteiro do aluno
29 de outubro de 2026

Roteiro do aluno — instalação e publicação

Disciplina: Laboratório de Programação 1 · **Professor:** Celso Gonsalez Saes
Projeto: Mercado Viva · **Objetivo:** publicar o sistema individualmente para avaliação por URL externa.

1. Entenda o que será instalado

O navegador acessa as páginas Flask hospedadas na Vercel. O código fica no GitHub e os dados persistentes ficam em um PostgreSQL externo, neste roteiro no Neon. Cada aluno cria um projeto e banco próprios. Não é necessário instalar PostgreSQL nem usar sudo no Debian.

O aplicativo usa **um banco relacional com 15 tabelas de negócio**, compartilhadas pelos módulos, para que uma venda atualize estoque, pagamentos e financeiro de forma consistente. Alembic acrescenta a tabela técnica de versão da estrutura. Não se cria um banco separado para cada módulo.

SQLite é oferecido exclusivamente para experimentar localmente, em um computador e sem uso concorrente. Em produção o aplicativo rejeita SQLite. Nunca use um arquivo .db ou /tmp como banco persistente da Vercel.

2. Extraia e confira a pasta

Baixe Supermercado_Mercado_Viva_Completo.zip. Selecione Extrair no gerenciador de arquivos. Abra o terminal dentro da pasta **supermercado**, onde estão app.py e requirements.txt.

Se preferir extrair pelo terminal, abra a pasta que contém o ZIP e execute:

```
python3 -m zipfile -e Supermercado_Mercado_Viva_Completo.zip .
cd supermercado
pwd
ls
python3 --version
```

Use Python 3.12 para reproduzir a versão definida em .python-version. Abra a pasta no VS Code pelo menu Arquivo > Abrir pasta, ou execute `code .` se o comando estiver disponível.

Estrutura principal

```
supermercado/
  app.py                        entrada Flask para Vercel
  manage.py                     administração local e carga inicial
  requirements.txt              dependências de execução
  requirements-dev.txt          dependências de testes
  vercel.json                   configuração da função
  .python-version               Python 3.12
  .env.example                  modelo sem credenciais
  .gitignore / .vercelignore    exclusões
  alembic.ini
  mercado/
    __init__.py                 fábrica, middleware e tratamento de erros
    db.py / models.py / security.py
    controllers/                auth, catalog, pos, reports
    services/operations.py      regras e transações
  templates/                    páginas HTML Jinja2
  public/css/aurora.css          tema roxo completo
  public/css/tropical.css        tema verde e laranja completo
  public/js/app.js               gráfico e interações
  migrations/                   versão inicial do esquema
  scripts/setup_local.py        gera segredo e .env local
  scripts/schema_postgresql.sql
  tests/                        testes isolados
  docs/                         roteiros e documentação
```

3. Prepare Python sem sudo

No terminal Bash do Debian, dentro de supermercado:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python scripts/setup_local.py
```

O script cria .env com segredo aleatório exclusivo; se .env já existir, preserva o arquivo. O comando `source` precisa ser repetido ao abrir outro terminal. Os pacotes são instalados na pasta .venv, sem modificar o Python do sistema.

Se o Debian não tiver venv/ensurepip ou não permitir baixar pacotes, a equipe técnica deverá preparar a imagem do laboratório ou disponibilizar um ambiente Python autorizado. Não use sudo nem pip global para contornar a restrição.

Alternativa no Windows PowerShell

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe scripts/setup_local.py
.\.venv\Scripts\python.exe -m alembic upgrade head
.\.venv\Scripts\python.exe manage.py create-admin
.\.venv\Scripts\python.exe manage.py create-operator
.\.venv\Scripts\python.exe manage.py seed
.\.venv\Scripts\python.exe app.py
```

Essa alternativa não precisa alterar a política de execução do PowerShell. Nos demais comandos deste roteiro, substitua `python` por `.\.venv\Scripts\python.exe` se não ativar o ambiente.

4. Teste local com SQLite

Este passo é opcional se você já vai usar PostgreSQL desde o início. Execute:

```
python -m alembic upgrade head
python manage.py create-admin
python manage.py create-operator
python manage.py seed
python app.py
```

Crie um administrador e um operador com e-mails identificáveis e **senhas diferentes**. O terminal não mostra caracteres ao digitar a senha: isso é esperado. Não há senha padrão. Use frases-senha com 15 a 128 caracteres, ao menos oito caracteres distintos, sem sequências previsíveis.

Acesse **http://127.0.0.1:5000**. O endereço local só funciona no próprio computador. Para encerrar o servidor, pressione Ctrl+C.

`seed` cria cinco categorias, um fornecedor fictício, 12 produtos e uma compra inicial com estoque. Por isso o financeiro começa com uma saída simulada: essa saída corresponde ao custo do estoque comprado, não a um erro. O comando se recusa a repetir a carga quando já há produtos.

5. Crie PostgreSQL externo no Neon

1. Acesse https://console.neon.tech e crie uma conta individual. Confirme os passos de verificação solicitados.
2. Crie um projeto com identificação do aluno. Consulte o plano, cotas e região disponíveis no painel; escolha uma oferta adequada à atividade.

3. Se o assistente já criou um banco, use-o. Caso precise criar outro, procure a seção de bancos/Databases e crie, por exemplo, **supermercado_aluno**. Selecione o banco correto ao obter a conexão.
4. Em **Connect**, copie a conexão PostgreSQL com pooling para **DATABASE_URL**. Copie também a conexão direta, sem pooling, para **DATABASE_DIRECT_URL**.
5. Preserve os parâmetros de TLS, como `sslmode=require`, e demais parâmetros fornecidos pelo provedor. Não remova o sufixo de pooling nem reconstrua host/senha manualmente.
6. Cole as conexões somente no .env local e nas variáveis privadas da Vercel. Não envie captura de tela contendo as credenciais.

Se você criar Neon pela aba Storage/Marketplace da Vercel em vez do console, conecte o banco ao projeto e confira os nomes das variáveis provisionadas. Este aplicativo lê DATABASE_URL. Para migrações locais, copie a conexão direta para DATABASE_DIRECT_URL; a integração pode chamá-la DATABASE_URL_UNPOOLED. Escolha **um** caminho de criação para não provisionar bancos duplicados.

Configure o .env local

Pare o servidor. Abra .env no VS Code e mantenha o SECRET_KEY já gerado, substituindo as conexões:

```
APP_ENV=development
SECRET_KEY=COLE_AQUI_O_SEGREDO_ALEATORIO_REAL
DATABASE_URL="postgresql://USUARIO:SENHA@HOST_POOLER/BANCO?sslmode=require"
DATABASE_DIRECT_URL="postgresql://USUARIO:SENHA@HOST_DIRETO/BANCO?sslmode=require"
APP_URL=http://127.0.0.1:5000
STORE_NAME="Supermercado Ana Silva"
```

As URLs acima são modelos, não conexões executáveis. Cole as URLs completas fornecidas pelo Neon. As aspas são úteis no arquivo .env; nos campos do painel Vercel, informe o valor sem aspas adicionais. O código normaliza postgres:// e postgresql:// para o driver psycopg.

6. Crie as tabelas e os dados no banco remoto

Com .env apontando para Neon, execute os comandos da pasta supermercado:

```
python -m alembic upgrade head
python manage.py create-admin
python manage.py create-operator
python manage.py seed
python manage.py check-db
python app.py
```

A migração usa DATABASE_DIRECT_URL; a aplicação e manage.py usam DATABASE_URL. **As duas conexões devem apontar para o mesmo banco e a mesma branch.** Os usuários e dados criados no SQLite não são transferidos automaticamente ao PostgreSQL. Recrie as contas e a carga nesse banco novo.

O primeiro comando cria todas as tabelas, chaves estrangeiras, índices e restrições. É seguro repetir `alembic upgrade head`: versões já aplicadas não são executadas novamente. Não execute `seed` repetidamente nem crie usuários duplicados.

O arquivo scripts/schema_postgresql.sql mostra o SQL completo da migração inicial. Ele é material de estudo e alternativa de instalação em banco vazio; **não execute o SQL manual e a migração como duas instalações diferentes**. A opção recomendada neste roteiro é Alembic.

Não crie uma rota pública /setup ou /create-admin e não use migrações como Build Command da Vercel. Execute os comandos administrativos pelo terminal, com as credenciais locais.

7. Faça a verificação antes de publicar

1. Entre como administrador e confira Produtos, Estoque, Compras, Financeiro e Usuários.
2. Saia e entre como operador. Abra um caixa com R$ 100,00.
3. Adicione SKU 1001 (arroz), quantidade 1, e SKU 5001 (leite), quantidade 2.
4. Total esperado da carga original: **R$ 39,70** (27,90 + 2 × 5,90).
5. Selecione dinheiro, informe R$ 50,00 recebidos e finalize. Troco esperado: **R$ 10,30**.
6. Imprima o ticket, confira estoque e relatório com o administrador.
7. Volte ao operador: dinheiro esperado no caixa **R$ 139,70**, se não houve outros movimentos. Feche com esse valor e confirme diferença zero.
8. Teste outras vendas em Pix, débito e crédito. Nenhum dado bancário deve ser solicitado.

8. Crie o repositório GitHub

1. Acesse https://github.com/signup ou entre na sua conta existente.
2. Crie um repositório privado chamado **nome-sobrenome-prontuario**. Exemplo: **ana-silva-sp123456b**.
3. Use minúsculas e hífens, sem acentos. Preserve todas as letras e números do prontuário, inclusive a letra final.
4. Crie o repositório vazio, sem arquivos automáticos se for enviar pelo terminal.

Se Git estiver instalado, execute na pasta supermercado, substituindo os exemplos:

```
git init
git config user.name "Nome do Aluno"
git config user.email "EMAIL_DA_CONTA_GITHUB"
git add .
git status
git diff --cached --name-only
git commit -m "Projeto final Mercado Viva"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/ana-silva-sp123456b.git
git push -u origin main
```

Antes do commit, confira que .env, .venv e arquivos .db não aparecem. A .env.example pode ser enviada. Não coloque token na URL; use o mecanismo de autenticação GitHub autorizado no computador. Se o remote origin já existir, confira `git remote -v` antes de alterar qualquer configuração.

Sem Git instalado

No navegador GitHub, use **Add file > Upload files** e envie os arquivos extraídos, mantendo a estrutura. Não envie somente o ZIP. Envie mercado, templates, public, migrations, scripts, docs e os arquivos da raiz. Não envie .env, .venv, banco local ou caches.

Crie arquivos ocultos com **Add file > Create new file** caso não sejam incluídos pelo envio: .gitignore, .vercelignore, .python-version e .env.example. Não é necessário usar GitHub Pages.

9. Importe na Vercel

1. Acesse https://vercel.com/signup e conecte sua conta GitHub.
2. Para a atividade individual não comercial, selecione o plano apropriado, como Hobby se disponível e adequado. Consulte as condições atuais.
3. Selecione **Add New > Project** e importe o repositório.
4. Se ele não aparecer, ajuste a integração GitHub e autorize somente o repositório desta atividade.
5. Configure **Project Name** como ana-silva-sp123456b (substitua pelo seu).
6. Framework: **Flask**, conforme a detecção da plataforma.
7. **Root Directory:** raiz, se app.py estiver na raiz do repositório; supermercado, se você enviou essa pasta como pasta interna.
8. Mantenha os padrões de Build Command e Output Directory. Não use `flask run`, `python app.py` ou migração como comando de build.
9. Production Branch: **main**.

O app.py exporta `app`, como exige o suporte Flask. Os ativos CSS e JavaScript estão em public e são referenciados como /css e /js.

10. Configure as variáveis de produção

Gere outro segredo no terminal, exclusivo para publicação:

```
python -c 'import secrets; print(secrets.token_hex(32))'
```

No painel do projeto, em **Environment Variables**, cadastre para **Production**:

Variável	Conteúdo
APP_ENV	production
SECRET_KEY	Segredo aleatório exclusivo gerado acima
DATABASE_URL	URL PostgreSQL com pooling, senha e TLS, copiada do Neon
APP_URL	https://ana-silva-sp123456b.vercel.app, ajustada ao endereço aceito
STORE_NAME	Supermercado Nome do Aluno

DATABASE_DIRECT_URL é usada para migração no terminal e não é necessária na execução web. Não há ADMIN_PASSWORD no código nem no build. O administrador já existe no PostgreSQL após o passo 6.

Para recuperação por e-mail, adicione opcionalmente SMTP_HOST, SMTP_PORT=587, SMTP_USER, SMTP_PASSWORD e SMTP_FROM conforme um provedor que suporte STARTTLS. Sem SMTP, a recuperação pelo responsável funciona com:

```
python manage.py reset-link
```

O comando solicita o e-mail e gera um link privado de uso único com validade de 20 minutos. No .env local, configure APP_URL com a URL publicada para o link apontar para a internet. Não inclua o link em commits nem na entrega pública.

Clique em **Deploy**. Não use banco de produção em previews de código de terceiros. Configure banco e segredos separados para Preview, ou avalie apenas Production; previews sem variáveis próprias podem não iniciar.

11. Configure a URL do aluno

1. Abra **Settings > Domains** no projeto Vercel.

2. Confira o endereço realmente associado à produção.
3. Se necessário, use a opção de editar/adicionar o domínio vercel.app e solicite **ana-silva-sp123456b.vercel.app**.
4. Se estiver ocupado, preserve nome e prontuário e acrescente sufixo: **ana-silva-sp123456b-lp1.vercel.app**.
5. Copie o endereço aceito pelo painel. Os exemplos não estão reservados.
6. Ajuste APP_URL para esse endereço em Production e faça **Redeploy**.
7. Abra a URL em janela anônima. O professor deve ver a tela de login do supermercado, sem precisar entrar em uma conta Vercel.

Um subdomínio vercel.app não exige compra nem configuração DNS externa. Um domínio próprio é opcional, pode ter custo e exige reproduzir os registros indicados no painel Vercel. Renomear o repositório não altera automaticamente o domínio.

Se aparecer autenticação da plataforma Vercel, revise **Deployment Protection** para a URL de produção que será avaliada. O login interno do supermercado continua obrigatório. Mantenha proteção dos previews conforme a necessidade.

12. Valide a persistência e entregue

- Faça uma venda fictícia na URL externa.
- Saia, entre novamente e confirme a mesma venda.
- Faça um redeploy sem trocar o banco e confirme que os dados continuam lá.
- Confirme o funcionamento dos temas, gráfico, Excel, ticket, compras e fechamento.
- Crie uma conta administrativa específica para o professor e uma conta de operador para testes, com senhas exclusivas.
- Preencha docs/ENTREGA_ALUNO_MODELO.md. Envie credenciais por canal privado indicado pelo professor, fora do repositório. Não envie SECRET_KEY nem conexão do banco.

Entregue a URL estável de produção, não a URL de commit/preview com hash. O professor entra em /login com a conta de avaliação. O acesso ao repositório privado só será necessário se também houver avaliação do código; conceda-o conforme a orientação da disciplina.

13. Atualize depois da publicação

```
git add .
git status
git commit -m "Melhorias do projeto final"
git push
```

A integração dispara nova publicação. Se você usar o navegador, faça commit na main. Mudanças apenas em HTML/CSS/JS não exigem migração.

Ao alterar modelos/tabelas, desenvolva e revise uma nova migração em banco de desenvolvimento:

```
python -m alembic revision --autogenerate -m "descricao_da_alteracao"
python -m alembic upgrade head
python -m pytest -q
```

Revise o arquivo gerado; autogenerate não entende toda intenção de mudança. Faça backup antes de migrar dados importantes e aplique a migração ao banco correto durante a atualização. Não edite a migração 0001 já aplicada nem use drop_all para atualizar.

14. Solução de problemas

Sintoma	Verificação
ModuleNotFoundError	Ative .venv e instale requirements.txt com o mesmo Python.
SECRET_KEY inválida	Gere o segredo e ajuste .env ou Production; faça redeploy se remoto.
Banco rejeitado na Vercel	DATABASE_URL precisa ser PostgreSQL; SQLite é somente local.
Tabela inexistente	Execute alembic upgrade head com conexão direta para o mesmo banco/branch.
Conexão recusada	Confira host, senha, banco, TLS e acesso de rede permitido no laboratório.
app.py não encontrado	Corrija Root Directory e envie os arquivos extraídos.
CSS ausente	Confira public/css e URLs /css/aurora.css e /css/tropical.css.
Login bloqueado	Aguarde 15 minutos após cinco falhas; confirme a conta criada no banco correto.

Sintoma	Verificação
Formulário expirado	Atualize a página; não reutilize formulário antigo após trocar sessão ou segredo.
Caixa fechado	Abra o caixa do usuário atual; o caixa de outro operador não serve.
Compra/venda duplicada	Confira o histórico antes de repetir; formulários possuem identificador único.
Erro 503	Confira banco, migração e Runtime Logs; a transação não deve ficar parcialmente gravada.
Gráfico vazio	Ajuste o período e faça uma venda concluída; canceladas não entram.
Financeiro negativo inicial	A carga seed registra a compra do estoque inicial.
Recuperação sem e-mail	Configure SMTP ou gere reset-link pelo terminal.
Plataforma pede login Vercel	Confira o domínio de produção e Deployment Protection.

Fontes oficiais para consulta

- Flask na Vercel: https://vercel.com/docs/frameworks/backend/flask
- Banco externo / Marketplace: https://vercel.com/docs/marketplace-storage
- Neon com SQLAlchemy: https://neon.com/docs/guides/sqlalchemy
- Neon, criação de conta: https://neon.com/docs/get-started/signing-up
- Variáveis de ambiente: https://vercel.com/docs/environment-variables
- Domínios: https://vercel.com/docs/domains/working-with-domains/add-a-domain
- GitHub e Vercel: https://vercel.com/docs/git/vercel-for-github
- Plano Hobby: https://vercel.com/docs/plans/hobby

Os nomes dos menus e as ofertas podem mudar. Consulte os painéis para cotas, disponibilidade e valores. O roteiro original anexado orientou o padrão de domínio, GitHub individual e PostgreSQL externo deste pacote.